

BAB 3 Komponen Dasar React Native

3.1 Tujuan Pembelajaran

1. Memahami konsep komponen dasar dalam React Native
2. Mengenali dan menggunakan komponen inti seperti *View*, *Text*, *Image*, dan *Button*
3. Mengimplementasikan komponen dasar React Native untuk membangun tampilan aplikasi sederhana
4. Mengombinasikan beberapa komponen dasar dalam satu tampilan aplikasi
5. Menyusun struktur antarmuka aplikasi mobile secara rapi dan terstruktur

3.2 Dasar Teori

3.2.1 Konsep Komponen dalam React Native

React Native menggunakan pendekatan berbasis komponen (*component-based architecture*), di mana tampilan aplikasi dibangun dari kumpulan komponen kecil yang dapat digunakan kembali (*reusable*). Setiap komponen merepresentasikan bagian tertentu dari antarmuka aplikasi.

Komponen dalam React Native berbeda dengan elemen HTML pada web. React Native menyediakan komponen inti (*core components*) yang akan diterjemahkan menjadi komponen native pada masing-masing platform (Android dan iOS).

3.2.2 Komponen Dasar React Native

Beberapa komponen dasar yang paling sering digunakan antara lain:

1. View

View merupakan komponen dasar yang berfungsi sebagai container atau pembungkus komponen lain, mirip dengan `<div>` pada HTML.

```
<View>
  <Text> 🖐️ Praktikum Pemograman Perangkat Bergerak </Text>
</View>
```

2. ScrollView

ScrollView adalah komponen yang digunakan untuk menampilkan konten yang dapat digulir (*scroll*) secara vertikal atau horizontal.

```
<ScrollView>
  {professors.map((item, index) => (
    <Text key={index}>
      {item}
    </Text>
  ))}
</ScrollView>
```

Komponen ini sangat berguna ketika jumlah konten melebihi tinggi atau lebar layar perangkat.

Syarat Penggunaan ScrollView:

- Hanya boleh memiliki satu parent component
- Seluruh konten harus dibungkus di dalam ScrollView
- Tidak disarankan untuk data yang sangat besar

3. ImageBackground

ImageBackground adalah komponen bawaan React Native yang digunakan untuk menampilkan gambar sebagai latar belakang (*background*) dari suatu tampilan.

```
<ImageBackground
  source={require("../assets/background.jpg")}
  resizeMode="cover" >
  <View>
    <Text>Praktikum Pemograman Perangkat Bergerak </Text>
  </View>
</ImageBackground>
```

Properti Penting ImageBackground:

Properti	Fungsi
source	Menentukan sumber gambar
resizeMode	Mengatur cara gambar menyesuaikan container
style	Mengatur ukuran dan layout
imageStyle	Mengatur style khusus gambar

Tabel 3.1 List Properti pada Imagebackground

4. StatusBar

StatusBar adalah komponen yang digunakan untuk mengatur tampilan status bar perangkat, yaitu area paling atas layar yang menampilkan informasi seperti waktu, baterai, dan sinyal jaringan.

```
<View>
  <StatusBar
    backgroundColor="#1e90ff"
    barStyle="light-content"
  />
  <Text>Welcome..</Text>
</View>
```

Properti Penting StatusBar:

Properti	Fungsi
backgroundColor	Warna latar (Android)
barStyle	Warna teks dan ikon
hidden	Menyembunyikan status bar
translucent	Status bar transparan

Tabel 3.2 List Properti pada StatusBar

Perbedaan Perilaku StatusBar di Android dan iOS:

Aspek	Android	iOS
backgroundColor	Bisa diatur	Tidak berlaku
barStyle	Berlaku	Berlaku
hidden	Berlaku	Berlaku

Tabel 3.3 Perbedaan Perilaku Statusbar di Android dan Ios

5. Text

Text digunakan untuk menampilkan teks pada aplikasi React Native. Semua teks wajib berada di dalam komponen Text.

```
<Text>👉 Praktikum Pemograman Perangkat Bergerak</Text>
```

6. Image

Image digunakan untuk menampilkan gambar, baik dari local asset maupun dari URL.

```
<Image
  source={{ uri: "https://reactnative.dev/img/tiny_logo.png" }}
  style={{ width: 100, height: 100 }}
/>

<Image
  style={{ width: 100, height: 100 }}
  source={require('./ibik-logo.png')}
/>
```

7. StyleSheet

StyleSheet digunakan untuk mengatur tampilan komponen agar lebih terstruktur dan konsisten.

```
const styles = StyleSheet.create({
  text: {
    fontSize: 16,
    fontWeight: "bold",
  },
});
```

8. TextInput

TextInput adalah komponen bawaan React Native yang digunakan untuk menerima input teks dari pengguna.

```
<TextInput
  placeholder="Nama lengkap"
  value={nama}
```

```
onChangeText={setNama}  
/>
```

Properti yang dimiliki TextInput:

Properti	Fungsi
placeholder	Menampilkan teks petunjuk
value	Menyimpan nilai input
onChangeText	Menangkap perubahan teks
keyboardType	Menentukan jenis keyboard
secureTextEntry	Menyembunyikan teks (password)

Tabel 3.4 Properti pada TextInput

9. Button

Button digunakan untuk membuat tombol interaksi sederhana.

```
<Button title="Klik Saya" onPress={() => alert("Tombol ditekan")} />
```

Perlu diperhatikan bahwa komponen Button tidak memiliki tampilan yang konsisten antar platform.

Keterbatasan Komponen Button

Beberapa keterbatasan utama:

- Tidak dapat mengatur warna background secara fleksibel
- Tidak dapat menambahkan icon
- Tidak dapat mengatur ukuran dan bentuk tombol
- Tampilan tidak seragam antara Android dan iOS

Oleh karena itu, untuk kebutuhan UI yang lebih fleksibel dan konsisten, disarankan menggunakan komponen alternatif.

Alternatif Button: Komponen Touchable

React Native menyediakan beberapa komponen Touchable yang digunakan untuk menangani interaksi sentuhan (*touch*) dan dapat dikombinasikan dengan komponen lain seperti *View* dan *Text*.

Komponen **Touchable** memungkinkan pengembang untuk:

- Membuat tombol dengan desain kustom
- Menjaga konsistensi tampilan antar platform
- Menambahkan ikon, warna, dan animasi

Jenis-Jenis Touchable

Komponen	Keterangan
TouchableOpacity	Memberi efek transparansi saat ditekan
TouchableHighlight	Memberi efek highlight
TouchableWithoutFeedback	Tanpa efek visual
Pressable	Lebih fleksibel dan modern

Tabel 3.5 Jenis-jenis Touchable

Contoh Penggunaan TouchableOpacity

```
<TouchableOpacity onPress={() => alert("Tombol ditekan")} >
  <Text>Simpan</Text>
</TouchableOpacity>
```

Penjelasan:

- TouchableOpacity berfungsi sebagai pendeteksi sentuhan
- View digantikan oleh TouchableOpacity
- Text digunakan untuk menampilkan label tombol
- Tampilan tombol dapat dikustomisasi sepenuhnya

Built-in Components React Native

Seluruh komponen dasar yang telah dibahas sebelumnya, mulai dari *View*, *Text*, *Image*, *ScrollView*, *TextInput*, *Button*, hingga komponen interaksi seperti *TouchableOpacity* dan *Pressable*, merupakan komponen bawaan (*built-in components*) yang disediakan langsung oleh **library react-native**. Komponen-komponen ini telah terintegrasi dengan sistem native Android dan iOS sehingga dapat digunakan tanpa perlu memasang library tambahan.

Untuk dapat menggunakan komponen tersebut, setiap komponen **harus di-import** terlebih dahulu dari **package react-native** sesuai kebutuhan. React Native tidak melakukan auto import, sehingga komponen yang tidak diimport tidak dapat digunakan di dalam file JSX.

Contoh penggunaan import komponen:

```
import { View, Text, StyleSheet } from "react-native";
```

Import dilakukan secara selektif, artinya hanya komponen yang digunakan saja yang perlu diimport. Pendekatan ini bertujuan untuk menjaga kode tetap rapi, mudah dibaca, dan efisien. Dengan memahami bahwa seluruh komponen dasar React Native merupakan bagian dari satu library utama, proses pengembangan aplikasi menjadi lebih terstruktur dan konsisten. Pengembang dapat fokus pada logika aplikasi dan desain antarmuka tanpa harus bergantung pada library eksternal untuk kebutuhan UI dasar.

Sumber: <https://reactnative.dev/docs/components-and-apis#basic-components>

3.2.3 Karakteristik Komponen Dasar

- Bersifat *cross-platform*
- Menggunakan JSX
- Dapat digabungkan satu sama lain
- Mendukung *styling* berbasis JavaScript

3.2.4 Aturan umum penggunaan komponen React Native

Dalam React Native, setiap komponen memiliki aturan penulisan (syntax rules) yang harus dipatuhi agar aplikasi dapat berjalan dengan baik. Aturan ini berkaitan dengan struktur JSX, pembungkusan komponen, dan penggunaan properti (props).

A. Komponen Harus Memiliki Tag Pembuka dan Penutup

Sebagian besar komponen React Native harus ditulis dengan tag pembuka dan penutup, terutama jika memiliki isi di dalamnya.

Code yang benar	Code yg salah
<code><Text>Hello</Text></code>	<code><Text>Hello</code>

Hal ini karena JSX bersifat **strict** seperti XML, sehingga setiap elemen harus ditutup dengan benar.

B. Self-Closing Component (Komponen Tanpa Isi)

Komponen yang tidak memiliki konten di dalamnya dapat ditulis dalam bentuk *self-closing*.

```
<Image source={require('./ibik-logo.png')} />
<Button title="Klik Saya" />
```

Jika tidak memiliki child (isi), maka cukup menggunakan tanda / di akhir tag.

C. Komponen Bertumpuk Harus Dibungkus oleh Satu Parent

Dalam JSX, satu komponen hanya boleh mengembalikan satu elemen induk (*parent*). Oleh karena itu, jika terdapat lebih dari satu komponen sejajar, semuanya harus dibungkus dalam satu komponen pembungkus.

Code yang benar	Code yg salah
<pre><View> <Text>Hello</Text> <Text>Febry</Text> </View></pre>	<pre><Text>Hello</Text> <Text>Febry</Text></pre>

Biasanya komponen *View* digunakan sebagai pembungkus (*container*).

D. Semua Teks Harus Berada di Dalam Komponen <Text>

React Native tidak mengizinkan teks berdiri sendiri seperti pada HTML.

Code yang benar	Code yg salah
<code><Text>Hello Dunia</Text></code>	Hello Dunia

E. Penulisan Nama Komponen Menggunakan Huruf Kapital

Komponen buatan sendiri (*custom component*) harus diawali dengan huruf kapital.

Code yang benar	Code yg salah
<code><Latihan1 /></code>	<code><latihan1 /></code>

Huruf kecil dianggap sebagai komponen bawaan/native.

F. Setiap List Harus Memiliki

Saat menggunakan perulangan seperti *map()* untuk menampilkan data, setiap elemen harus memiliki *prop* key.

```
{data.map((item, index) => (
  <Text key={index}>
    {item}
  </Text>
))}
```

G. Style Ditulis dalam Bentuk Object

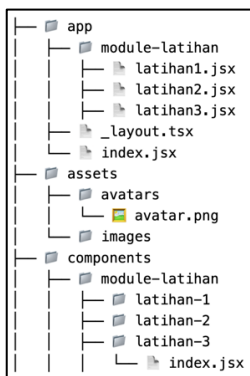
Tidak seperti CSS, React Native menggunakan object JavaScript untuk styling.

```
<Text style={{ color:'blue' }}>Warna Biru</Text>
```

3.3 Praktikum

Pada latihan praktikum ini, dilakukan pembuatan tampilan form profil sederhana dengan memanfaatkan berbagai komponen dasar React Native yang telah dipelajari sebelumnya. Form ini dirancang untuk melatih pemahaman dalam menyusun antarmuka aplikasi mobile secara terstruktur, interaktif, dan sesuai dengan kebutuhan dasar aplikasi nyata.

Melalui latihan ini, diharapkan pemahaman terhadap penggunaan komponen seperti Text, TextInput, View, Image, dan komponen lainnya dapat meningkatkan interaksi, serta mampu mengintegrasikan berbagai komponen tersebut dalam satu tampilan form yang sederhana namun fungsional.



Langkah 1: Membuat RFC dengan nama "Latihan 3"

Buatlah navigasi dan file RFC bernama Latihan 3, dan tampilkan tulisan "Memulai Tugas Latihan Praktikum 3",

Langkah 2: Menambahkan asset image

Carilah gambar yang mewakili sebuah avatar atau sejenisnya. Lalu taruh kedalam folder assets, seperti contoh pada gambar disamping, dimana sample gambar bernama **avatar.png** berada pada *assets* → *avatars*.

Gambar 3.1 Struktur Folder Project ditambah Assets

Langkah 3: Merancang struktur code JSX Latihan 3

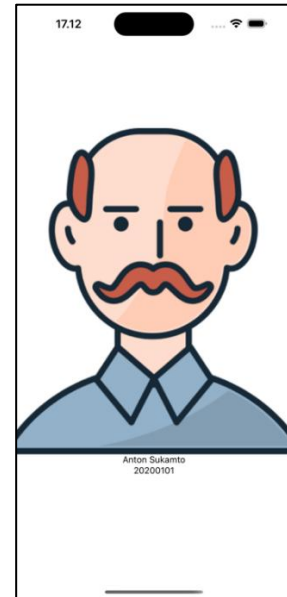
1. Mempersiapkan informasi data diri dengan tipe data Object, dan menampilkan komponen View, Text, Image dan StyleSheet.

Code	Output
------	--------

```

1 import { View, Text, StyleSheet, Image } from "react-native";
2 import React from "react";
3
4 export default function Latihan3() {
5   const personalData = {
6     name: "Anton Sukanto",
7     nim: "20200101",
8     address: "Jl. Rangka Gading No.01, Gudang, Kecamatan Bogor Tengah, Kota Bogor",
9     email: "anton.sukanto@ibik.ac.id",
10    phone_number: "081234567890",
11  };
12  return (
13    <View style={styles.container}>
14      <Image source={require("../assets/avatars/avatar.png")} />
15      <Text>{personalData.name}</Text>
16      <Text>{personalData.nim}</Text>
17    </View>
18  );
19 }
20
21 const styles = StyleSheet.create({
22   container: {
23     flex: 1,
24     backgroundColor: "#fff",
25     alignItems: "center",
26     justifyContent: "center",
27     paddingHorizontal: 20,
28   }
29 });
30

```



Gambar 3.2 Output program Latihan 3

Di dalam komponen Latihan3, terdapat sebuah variabel bernama `personalData` yang bertipe object. Objek ini digunakan untuk menyimpan beberapa informasi pribadi, seperti nama, NIM, alamat, email, dan nomor telepon. Penggunaan object pada konteks ini bertujuan untuk mengelompokkan data yang saling berkaitan agar lebih terstruktur dan mudah dikelola.

Pengaturan tampilan dilakukan menggunakan `StyleSheet.create`, di mana style container menerapkan konsep Flexbox. Properti `flex: 1` memastikan tampilan memenuhi layar, `alignItems: "center"` dan `justifyContent: "center"` digunakan untuk memposisikan konten ke tengah layar, sedangkan `paddingHorizontal` memberikan jarak horizontal agar tampilan lebih rapi.

Pada bagian tampilan (JSX), komponen `View` digunakan sebagai container utama yang membungkus seluruh elemen antarmuka. Untuk menampilkan gambar profil, digunakan komponen `Image` dengan properti `source` yang memuat gambar lokal dari folder `assets`. Gambar ini berfungsi sebagai ilustrasi avatar pengguna.

Selanjutnya, komponen `Text` digunakan untuk menampilkan nilai nama dan NIM yang diambil dari object `personalData`. Pemanggilan data dilakukan menggunakan tanda kurung kurawal `{}`, yang menandakan bahwa nilai tersebut merupakan variabel JavaScript yang dievaluasi di dalam JSX.

2. Pengaturan Tampilan Komponen Menggunakan StyleSheet

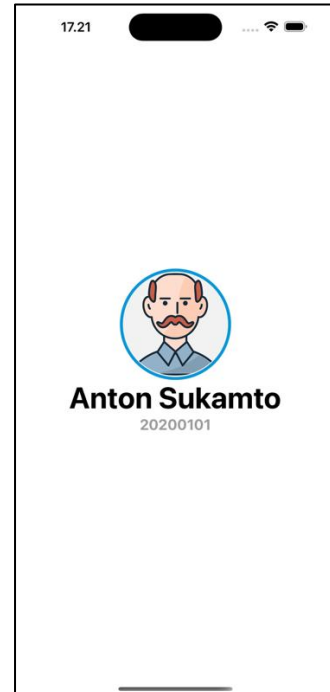
Pada tahap ini, dilakukan pengaturan tampilan komponen `Image` dan `Text` menggunakan `StyleSheet` agar antarmuka aplikasi terlihat lebih rapi, proporsional, dan mudah dibaca. Penggunaan `StyleSheet` memungkinkan pemisahan antara logika aplikasi dan pengaturan tampilan sehingga kode menjadi lebih terstruktur dan mudah dipelihara.

Code

Output

a. Tambahkan code ini pada bagian styleSheet:

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: "#fff",
    alignItems: "center",
    justifyContent: "center",
    paddingHorizontal: 20,
  },
  headers: {
    title: {
      fontWeight: "bold",
      fontSize: 40,
    },
    subtitle: {
      fontWeight: "bold",
      fontSize: 20,
      color: "#9b9d9f",
    },
  },
  img_avatar: {
    width: 150,
    height: 150,
    borderRadius: 100,
    borderColor: "#0996d7",
    borderWidth: 4,
    padding: 2,
    backgroundColor: "#f2f2f2",
  },
});
```



Gambar 3.3 Output program Latihan 3

b. Selanjutnya tambahkan properti style pada fragment JSX Image dan Text:

```
<View style={styles.container}>
  <Image
    source={require("../assets/avatars/avatar.png")}
    style={styles.headers.img_avatar}
  />
  <Text style={styles.headers.title}>
    {personalData.name}
  </Text>
  <Text style={styles.headers.subtitle}>
    {personalData.nim}
  </Text>
</View>
```

3. Perancangan Form Isian Profil Menggunakan TextInput dan Touchable

Pada tahap ini, dilakukan pembuatan form isian profil sederhana yang terdiri dari nomor telepon, email, dan alamat menggunakan komponen TextInput. Untuk menangani aksi penyimpanan data, digunakan komponen Touchable sebagai tombol Save. Latihan ini bertujuan melatih penggunaan komponen input dan interaksi sentuhan dalam membangun antarmuka form yang sederhana dan terstruktur.

Code

Output

a. Membuat struktur isian phone number dengan TextInput, tambahkan code dibawah ini:

```
<View style={styles.container}>
  <Image
    source={require("../assets/avatars/avatar.png")}
    style={styles.headers.img_avatar}
  />
  <Text style={styles.headers.title}>{personalData.name}</Text>
  <Text style={styles.headers.subtitle}>{personalData.nim}</Text>

  <View>
    <View>
      <Text>Phone</Text>
      <TextInput
        value={personalData.phone_number}
        autoFocus
        keyboardType="numeric"
      />
    </View>
  </View>
</View>
```

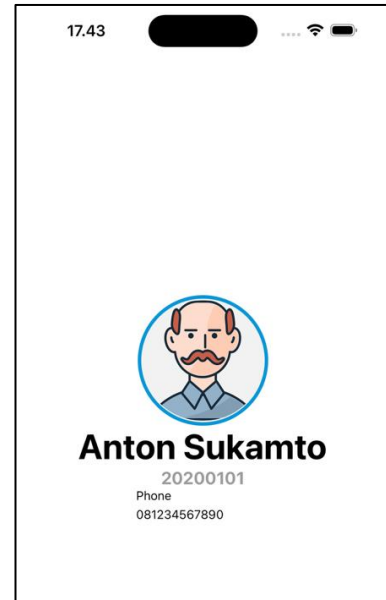
add new code

b. Buatlah style tambahan untuk mendesain isian dari TextInput:

```
identity: {
  container: {
    alignSelf: "stretch",
    padding: 10,
    marginTop: 20,
  },
  card_input: {
    borderWidth: 1,
    borderColor: "#9b9d9f",
    borderRadius: 10,
    paddingHorizontal: 10,
    paddingVertical: 5,
    marginBottom: 10,
  },
},

title: {
  color: "#9b9d9f",
  fontSize: 16,
  marginBottom: 0,
},
input_text: {
  color: "#000",
  fontSize: 16,
  padding: 0,
},

button: {
  alignItems: "center",
  backgroundColor: "#0ea6d0",
  padding: 15,
  borderRadius: 10,
},
button_text: {
  fontSize: 18,
  color: "white",
  fontWeight: "bold",
},
});
```



Gambar 3.4 Output program Latihan 3

Code

Output

c. Tambahkan style yang telah ditambahkan tadi kedalam fragment JSX:

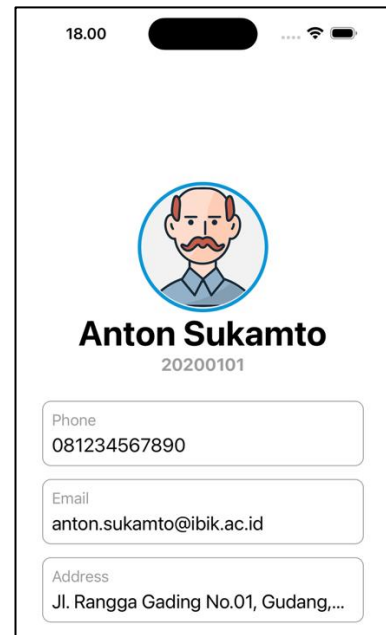
```
<View style={styles.identity.container}>
  <View style={styles.identity.card_input}>
    <Text style={styles.identity.title}>Phone</Text>
    <TextInput
      value={personalData.phone_number}
      style={styles.identity.input_text}
      autoFocus
    />
  </View>
</View>
```

d. Lakukan hal yang sama seperti point (c), untuk isian email dan address. Dengan mengkopi code dari *View.identity.card_input* hingga penutupnya, dan mengganti isian properti value pada TextInput dengan key objek yang disesuaikan.

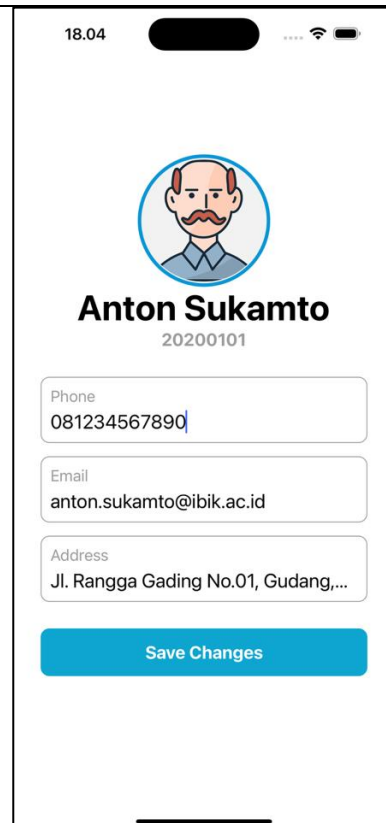
e. Selanjutnya ialah membuat sebuah Tombol sebagai aksi untuk menyimpan isian dengan memanfaatkan komponen *TouchableOpacity*.

f. Tambahkan code tombol "Save Changes" seperti dibawah ini, masih didalam komponen pembungkus *View.identity.container*:

```
<View style={{ marginTop: 15 }}>
  <TouchableOpacity
    style={styles.identity.button}>
    <Text
      style={styles.identity.button_text}>
      Save Changes
    </Text>
  </TouchableOpacity>
</View>
```



Gambar 3.5 Output program Latihan 3



Gambar 3.6 Output program Latihan 3

Melalui latihan praktikum ini, telah diterapkan berbagai komponen dasar React Native untuk membangun tampilan form profil sederhana yang fungsional. Latihan ini diharapkan dapat memperkuat pemahaman dalam penggunaan komponen input, pengelolaan interaksi pengguna, serta penyusunan antarmuka yang rapi dan terstruktur. Konsep yang dipelajari pada latihan ini menjadi fondasi penting untuk pengembangan fitur form yang lebih kompleks pada aplikasi mobile di tahap selanjutnya.

3.4 Tugas Latihan Praktikum

Latihan 1: Melanjutkan Screen Latihan 3

Jika telah mengerjakan Latihan praktikum 3, diharuskan menambahkan beberapa isian data personal minimal 6 buah data (sudah termasuk `phone_number`, email dan address). Dimana masing-masing data personal tersebut memiliki tipe data yang berbeda-beda (number, array, object, boolean, dan lain-lain).

Latihan 2: Memanfaatkan Komponent Container Lainnya

Pada pembungkus utama aplikasi gantilah menggunakan komponen container `<ScrollView>`, lalu tambahkan container `<ImageBackground>`. Setelah anda mencoba beberapa container tersebut, berikan pendapat apa perbedaan antara pembungkus container `<View>`, `<ScrollView>` dan `<ImageBackground>`.

Latihan 3: Laporan Singkat

1. Buatlah dokumentasi dalam mengerjakan soal Latihan 1 hingga Latihan 2, dengan memberikan Screenshot baik dari sisi aplikasi ataupun code yang sudah dimodifikasi dengan penjelasan dan alur yang baik dan benar. Dokumentasi disimpan dalam bentuk PDF dan ditaruh dalam project react expo yang telah anda kerjakan.
2. Upload project tersebut pada GITHUB yang telah dibuat sebelumnya di repository "**PPB-Praktikum-NPM**", namun *commit* dengan nama "**Tugas Praktikum-3**".
3. Pengumpulan Tugas Latihan Praktikum dikumpulkan selama 1 minggu kedepan sejak hari ini (sesuai dengan jadwal praktikum).